

Importing Libraries



```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.cluster import KMeans
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import confusion_matrix
```

- **Pandas** → For handling `.csv` data & numerical arrays.
- **Matplotlib & Seaborn** → For creating the statistical graphs.
- **Scikit-Learn (sklearn) :**
 1. **KMeans** → The *Clustering* Algorithm.
 2. **StandardScaler** → *Normalizes* the data (*crucial for K-Means*). Scales the data to be treated as equally.
 3. **confusion_matrix** : Used at the end to check *how accurate* the clustering was compared to the true labels.

Data Preparation



```
1 # Load the dataset & rename columns for accessibility
2 df = pd.read_csv('Dataset.csv')
3 df.columns = ['Temperature', 'Luminosity', 'Radius', 'Absolute_magnitude',
               'Star_type', 'Star_color', 'Spectral_class']
```

- **Data Reading** → `pd.read_csv('Dataset.csv')` reads the data & stores its info. as `df`.
- **Renaming** → The columns are renamed for *ease* of human reading using `df.columns`.

Convert to Numeric Values



```
1 # Convert to numeric values(i.e.'float' or 'int')
2 df = df.apply(pd.to_numeric, errors="coerce")
```

- `pd.to_numeric` → Try to convert all the values in the **columns** into numbers (*integers* or *floats*).
 - `errors="coerce"` → Replaces any invalid data with ' *NaN* ' (missing values).
-

Dropping the Missing Values



```
1 # Dropping rows with missing values(i.e. skipping them)
2 df.dropna(subset=['Temperature', 'Absolute_magnitude'], inplace=True)
```

- It *Removes/Passes* the rows that have missing values so that the plot doesn't break.
 - `inplace` :
 1. `inplace=True` → It **modifies** the original DataFrame directly (*no copy*).
 2. `inplace=False` (*Default*) → It returns a **new** DataFrame, leaving the *original unchanged*.
-

Features Selection



```
1 # Select features for HR diagram (Temperature & Absolute Magnitude)
2 features = df[['Temperature', 'Absolute_magnitude']]
```

- The **HR Diagram** is strictly a plot of *Temperature vs. Absolute Magnitude*. Therefore, only these two columns are selected as features.
-

Scaling



```
1 # Scale the features (K-Means is sensitive to the scale of data)
2 scaler = StandardScaler()
3 features_scaled = scaler.fit_transform(features) # <class 'numpy.ndarray'>
```

Working of StandardScaler

It *squishes* both features into a similar range (*mean=0, variance=1*) so the algorithm treats them equally. It is because :

1. **Temperature** ranges from *2,000 to 40,000+ Kelvin*.

2. **Absolute Magnitude** ranges from *-10 to +20*.

Working of `fit_transform`

Step A: `fit`

The scaler looks at your data and calculates two key statistics for every column:

1. **Mean (μ)** : The average value.
2. **Standard Deviation (σ)** : How spread out the data is.

Step B: `transform`

It applies the **Standardization** (or *Z-score normalization*) formula to every single data point:

$$z = \frac{x - \mu}{\sigma}$$

Creating the Canvas `plt.subplots`



```
1 # Perform K-Means Clustering & Visualization for different values of k
2 ks = [3, 4, 5, 6]          # Testing k=3 (Main Sequence, Giant, White Dwarf) &
  others
3 fig, axes = plt.subplots(1, len(ks), figsize=(20, 6))
```

- `ks` is list of all values of k (*k = No. of Clusters*).
- `1, len(ks)` tells **Matplotlib** to create *1 row* and `len(ks)` *columns* of plots.
- `figsize=(20, 6)` sets the appropriate size for the figure.
- `axes` is an **array** that holds each of these *three blank plots*, allowing us to draw on them one by one using a loop.

The Clustering Loop

Code of Whole Loop



```
1 for i, k in enumerate(ks):
2     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
3     clusters = kmeans.fit_predict(features_scaled)  # <class
  'numpy.ndarray'>
```

```

4     df[f'Cluster_{k}'] = clusters
5     centroids_scaled = kmeans.cluster_centers_
6
7     # Transform centroids back to original scale for plotting
8     centroids = scaler.inverse_transform(centroids_scaled)
9
10    # Plot clusters
11    sns.scatterplot(data=df, x='Temperature', y='Absolute_magnitude',
12                    hue=f'Cluster_{k}', palette='viridis', ax=axes[i], s=60)
13
14    # Mark Centroids
15    axes[i].scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X',
16                    s=200, label='Centroids', edgecolors='black')
17
18    # HR Diagram conventions: Invert Temperature (X) & Magnitude (Y)
19    axes[i].invert_xaxis() # Values go from high to low
20    axes[i].invert_yaxis()
21    axes[i].set_title(f'K-Means HR Diagram (k={k})')
22    axes[i].set_xlabel('Temperature (K)')
23    axes[i].set_ylabel('Absolute Magnitude (Mv)')
24    axes[i].legend()

```

The For Loop

`enumerate(ks)` → This gives us both the index (`i`) and the value (`k`).

- When `k=3`, `i=0`.
- When `k=4`, `i=1`.
- When `k=5`, `i=2`.
- When `k=6`, `i=3`.

Training the Data (Running K-Means)



```

1 kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
2 clusters = kmeans.fit_predict(features_scaled)

```

- `n_clusters=k` → Tells the computer how many *clusters* to form.
- `random_state=42` → K-Means starts by placing '*centroids*' (center points) randomly. Setting a *seed* (42) ensures that every time you run the code, the *random start is the same*, making your results reproducible.
- `n_init=10` → The *algorithm will run 10 times* with different random starting points and pick the best result to avoid getting stuck in a "bad" random start. It gives more *accuracy*.

- `kmeans.fit_predict()`:
 1. `fit` → The algorithm starts the **K-Means** process.
 2. `predict` → The algorithm gives every star cluster a label (*0 to k-1*) for the finalized clusters.

Handling Centroids



```
1 df[f'Cluster_{k}'] = clusters
2 centroids_scaled = kmeans.cluster_centers_
3
4 # Transform centroids back to original scale for plotting
5 centroids = scaler.inverse_transform(centroids_scaled)
```

- `df[f'Cluster_{k}'] = clusters` → K-Means has finished grouping the stars, but those labels (0, 1, 2...) are currently just sitting in a separate list in the computer's memory. This line takes those labels and sticks them back into your original table (`df`) as a *new column* `f'Cluster_{k}'`.
- `cluster_centers_` → It returns an **array** of coordinates of *Centroids*.
- `inverse_transform` → It *scales* the coordinates *back* into real-world units.

The Math → Earlier, we subtracted the mean and divided by the standard deviation.

`inverse_transform` does the opposite, it multiplies by the standard deviation and adds the mean back.

Plotting the HR Diagram



```
1 # Plot clusters
2 sns.scatterplot(data=df, x='Temperature', y='Absolute_magnitude',
3                 hue=f'Cluster_{k}', palette='viridis', ax=axes[i], s=60)
4
5 # Mark Centroids
6 axes[i].scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X',
7                 s=200, label='Centroids', edgecolors='black')
8
9 # HR Diagram conventions: Invert Temperature (X) & Magnitude (Y)
10 axes[i].invert_xaxis() # Values go from high to low
11 axes[i].invert_yaxis()
12 axes[i].set_title(f'K-Means HR Diagram (k={k})')
13 axes[i].set_xlabel('Temperature (K)')
```

```

12 axes[i].set_ylabel('Absolute Magnitude (Mv)')
13 axes[i].legend()

```

- `centroid[:, x]` → This syntax follows the rule: `array[rows, columns]`

1. The Colon (`:`): This means 'Take All Rows.'

2. The Number (`0` or `1`): This specifies which column to grab.

`centroid[:, 0]` → This pulls out all the **Temperature** values for the *centers*.

`centroid[:, 1]` → This pulls out all the **Absolute Magnitude** values for the *centers*.

- `axes[i]` → i^{th} subplot.
- `axes[i].legend()` → It adds a **key** or **label box** to your graph.

Showing & Saving our Plot



```

1 plt.tight_layout()
2 plt.savefig('HR Diagram Clusters.png') # Saves the plot
3 plt.show()

```

- `plt.tight_layout()` is a "cleanup" command. Its job is to automatically adjust the **padding** and **spacing between** your **subplots** so that everything fits neatly within the figure window.

Confusing Matrix



```

1 # Evaluation: Confusion Matrix (comparing clusters to actual labels)
2 for k in ks:
3     print(f"\n💻 Confusion Matrix ( Actual Type vs Cluster Label for k=
      {k} ) :\n")
4     print(confusion_matrix(df['Star_type'], df[f'Cluster_{k}']))

```

- The dataset actually contains **k real star types**.
- The `confusion_matrix` compares: **True Types** (*rows*) vs. the **K-Means Cluster Labels** (*columns*).
- **Goal** → A perfect clustering would result in a matrix where most numbers are on the *diagonal* (or *permuted diagonal*).

Saving the results as csv file



```
1 # Save the results to CSV
2 df.to_csv('Cluster Star Data.csv', index=False)
```